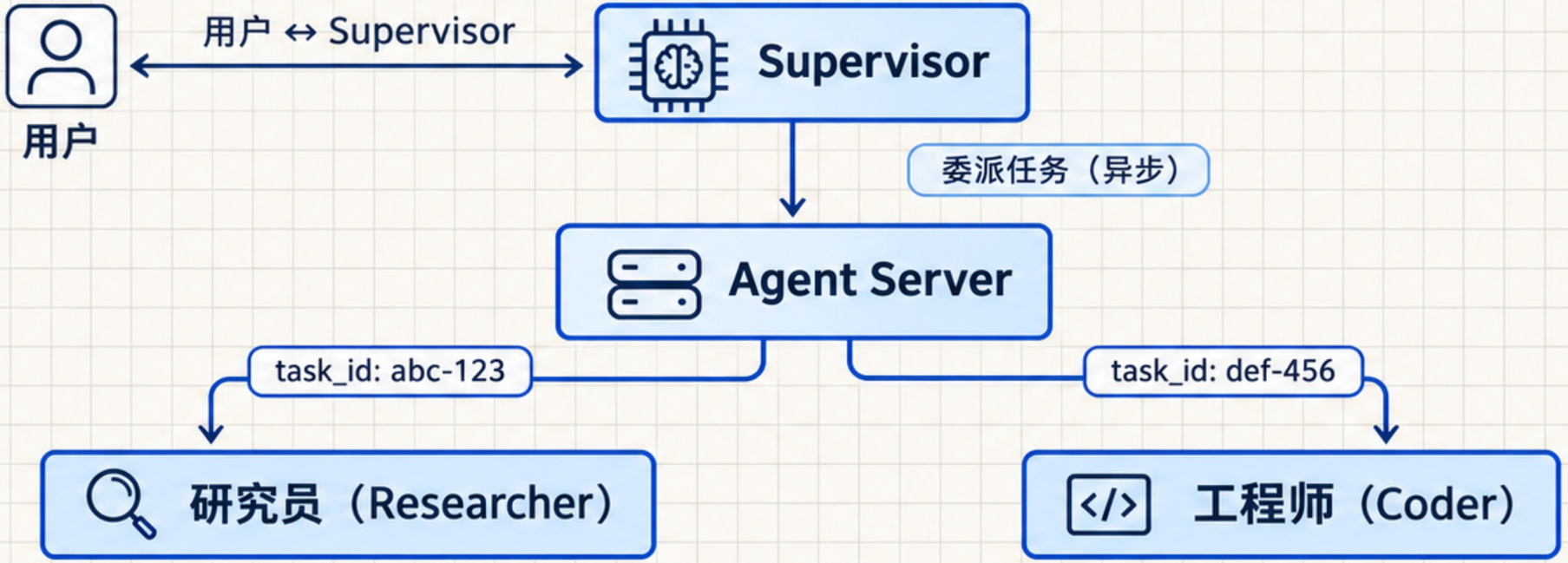
校准基线0.7.22 · 预览能力，版本需锁定

Deep Agents 实战

第6章：异步子 Agent 与后台任务控制

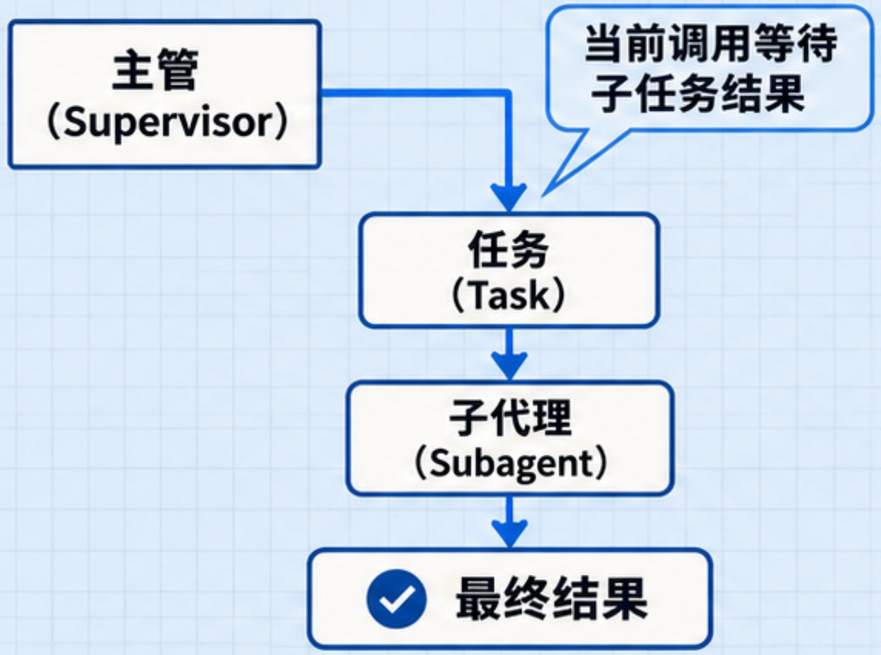


ⓘ | 启动后返回任务 ID，后台继续执行。

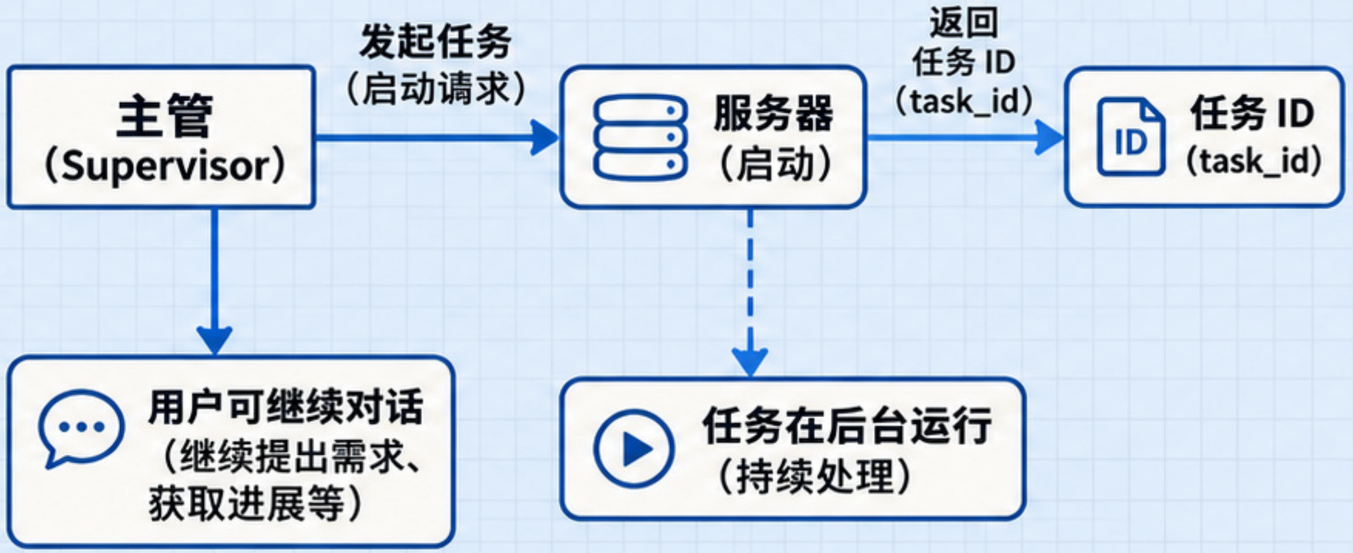
⚠ | 非零耗时，不保证无限并发。

同步等待与异步委派：改变的是控制方式

同步（等待完成）



异步（发起后继续互动）



⚠ 同步 task 不提供本章的五个任务管理工具；不要把它说成整个系统绝对无法取消。

⚠ 异步也需要等待“启动请求”的处理（如校验、排队、创建任务等），并非零延迟。

💡 **长程且需要互动控制时考虑异步；5秒只是经验，不是 API 门槛。**

先分清：协议、运行服务、追踪

Agent Protocol

- thread 与 run 的 API 约定。

Agent Server

- 加载 graph、调度后台 run、保存状态。

Observability

- 记录与分析 trace，不负责启动服务。

thread	= 会话状态，⚠ 不是操作系统线程。
run	= 该会话上的一次执行。



本地 langgraph dev 可起步，不要求先上云部署。

AsyncSubAgent 只是配置，不会启动服务

```python
from deepagents import AsyncSubAgent, create_deep_agent

worker = AsyncSubAgent(
    name="researcher",
    description="多源研究并返回摘要",
    graph_id="researcher",
)

agent = create_deep_agent(
    model=model, subagents=[worker],
)
```

必填配置（3项）

name / description / graph_id

可选：url

服务地址；省略时须有同部署 ASGI 服务，**且使用异步入口。**

可选：headers

自定义请求头（如鉴权信息）。

- **model 是已配置模型；graph_id 对应服务已注册 graph 或 assistant。**
- **省略 url 须有同部署 ASGI 服务，且使用异步入口。**

本地起步：先注册，再启动，再调用

1 注册：编写 langgraph.json

```
{
  "dependencies": ["./"],
  "graphs": {
    "supervisor": "./graphs/supervisor.py:graph",
    "researcher": "./graphs/researcher.py:graph"
  },
  "env": "./.env"
}
```

2 启动：运行开发服务

```
langgraph dev --n-jobs-per-worker 4
```

3 调用：按链路发起请求

客户端SDK → supervisor → ASGI → researcher

⚠ ASGI用异步入口（如ainvoke），不是同步invoke。

ⓘ 本地in-memory开发不要求LangSmith Key；模型Key仍需配置。服务提供检查点组件，导出graph时不手动添加checkpointer。

五个工具：参数与职责要准确

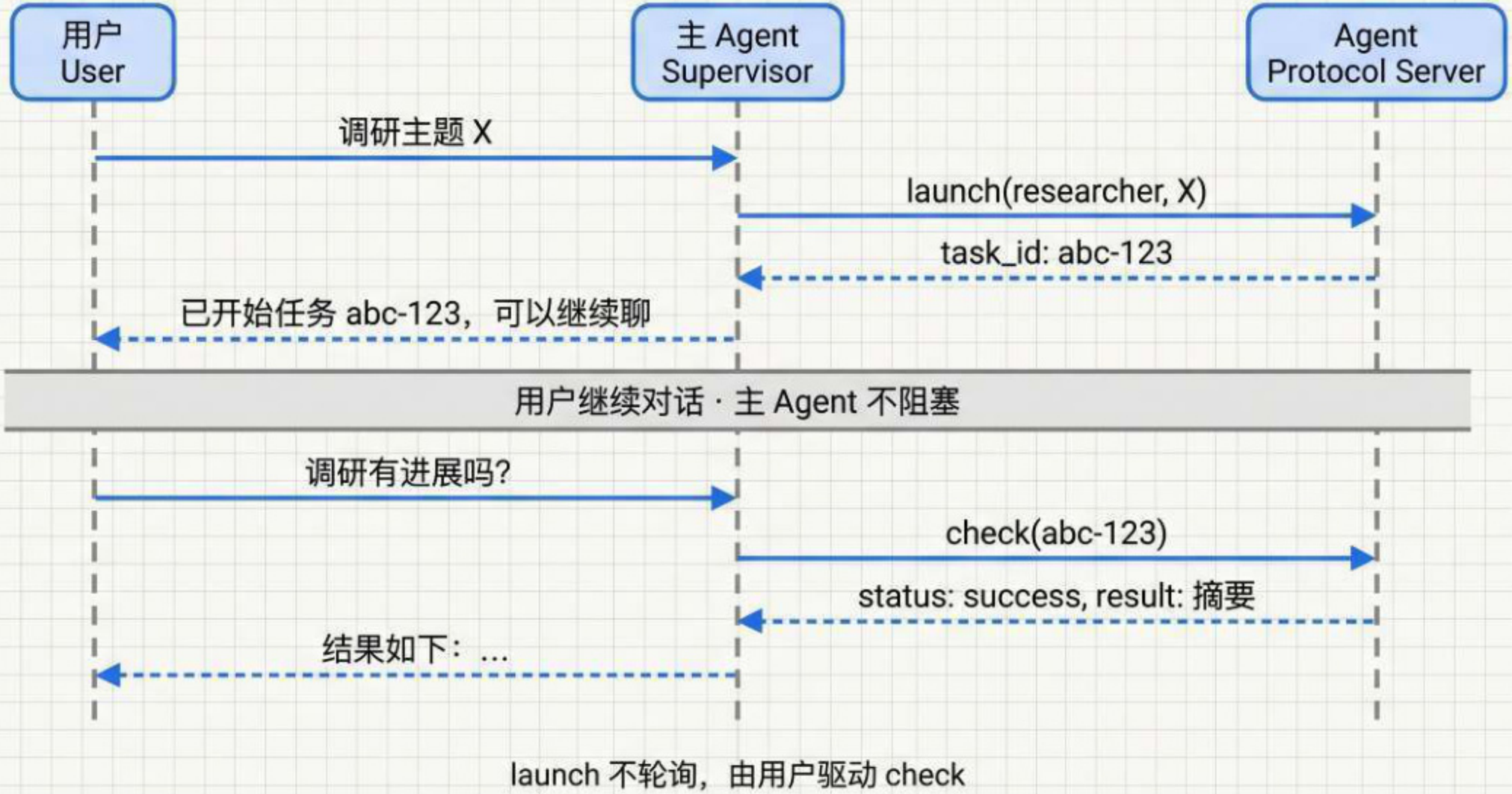
启动异步任务	start_async_task(description, subagent_type)	新建thread+run，返回task_id。
查询异步任务	check_async_task(task_id)	查询服务端run，success时读取最终输出。
更新异步任务	update_async_task(task_id, message)	同thread发起新run，interrupt策略。
取消异步任务	cancel_async_task(task_id)	请求取消当前run，本地标记cancelled。
列出异步任务	list_async_tasks(status_filter="all")	汇总当前state跟踪任务。

⚠ task_id等于子任务thread_id；使用完整ID。

⚠ 工具可失败，返回ID不等于任务已完成。

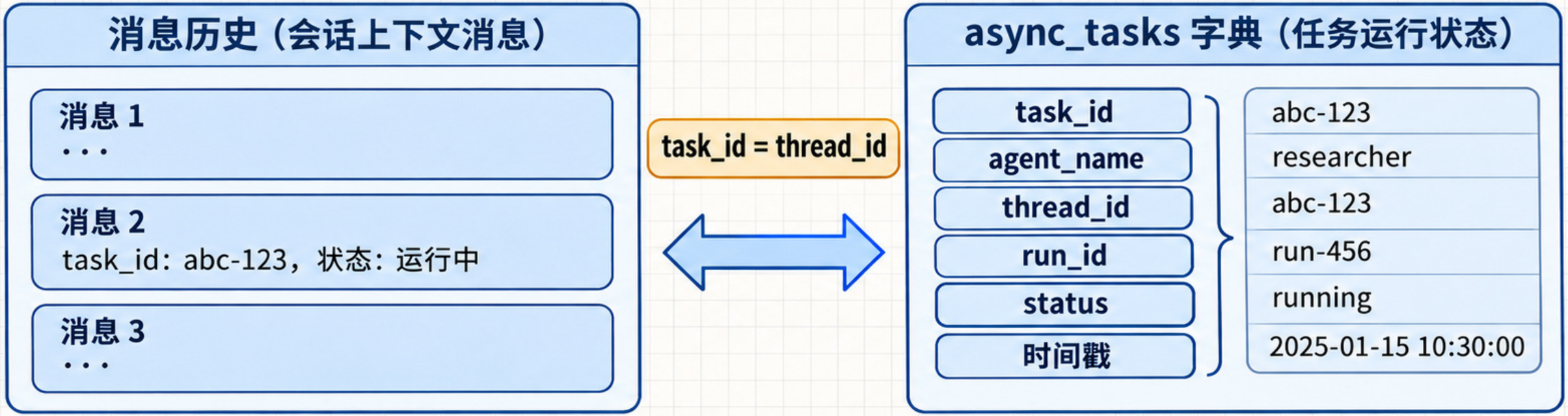
一次完整生命周期

User · Supervisor · Agent Protocol Server



launch 不轮询，由用户驱动 check

async_tasks：独立状态，不是永久保险箱



⚠ 默认摘要不删除此字段。

⚠ 跨轮需恢复同一主Agent thread 状态。

⚠ 跨进程需服务持久化，不能保证永不丢失。

list_async_tasks 只列当前 state 已跟踪任务，不是服务端全局任务搜索。

更新与取消：不是原地改写，也不是撤销

理解更程（update）与取消（cancel）的执行流程与语义

更新（update）：在原thread追加输入并创建新run

流程：在同一thread中对任务执行 update(task_id, message)，在原thread追加输入并创建新的run。

同一thread/task_id
在原thread中进行更新

↓

interrupt策略
中断当前运行

↓

新run_id
创建新的run并继续运行

取消（cancel）：通过服务端取消请求并确认实际状态

流程：发起取消请求，由服务端处理取消；本地先标记为 cancelled，随后通过 check_async_task 确认实际run状态。

cancel_async_task(task_id)
发起取消请求

↓

请求取消run
由服务端处理取消

↓

本地cancelled
本地先标记为 cancelled

↓

check_async_task(task_id)
确认实际run状态

⚠ **注意：**外部副作用可能已发生，取消不等于回滚。

关于 list：对运行的终态使用缓存，不能单凭 list 确认取消已生效。

两种传输，三种部署拓扑

传输方式

ASGI（进程内传输）



⚠ 异步入口，减少网络传输（而非零耗时）

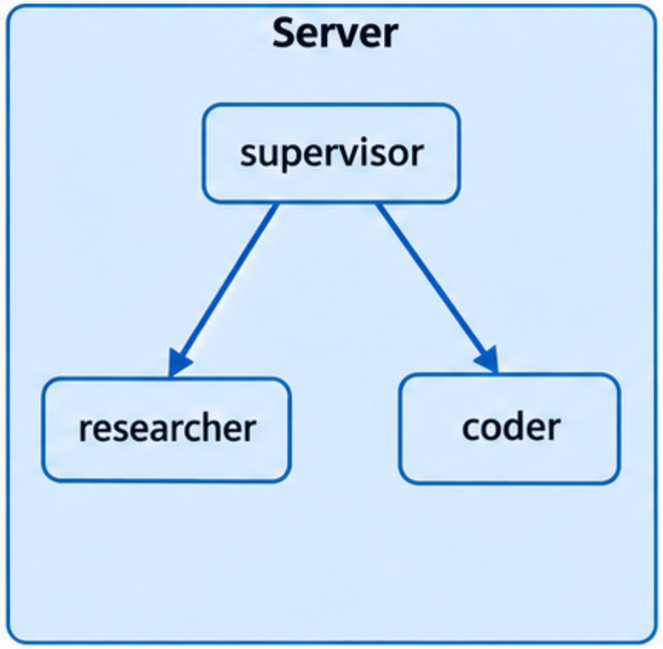
HTTP（网络传输）



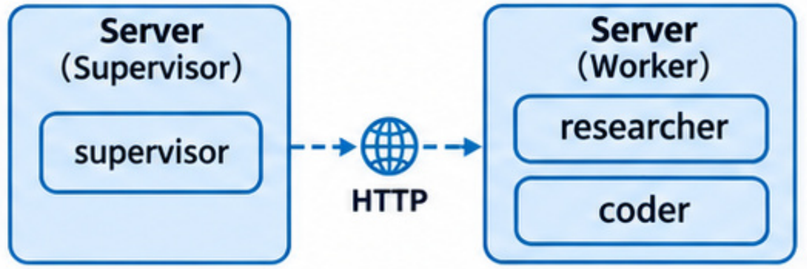
⚠ 传 url，按服务配置鉴权；
⚠ 部署 Key 或 headers。

部署拓扑

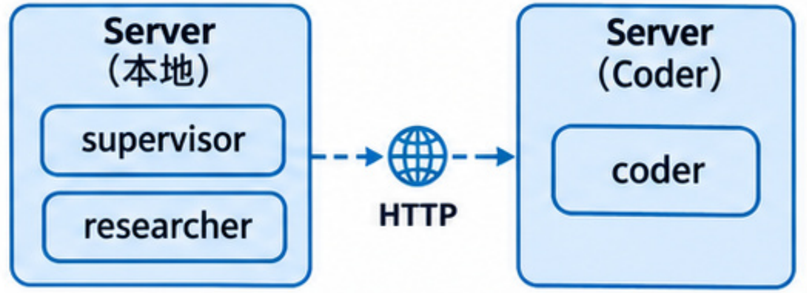
单部署（同一 Server）



拆分部署（Supervisor Server → HTTP → Worker Server）



混合部署（本地 Researcher + HTTP 连接 Coder）



⚠ 同部署仍需 Server；模型等外部请求仍可能走网络。先单部署，按资源/团队边界拆分。

状态与排障：报告已核实的事实

1	状态查询	check读当前run；success不等于内容质量合格。
2	list	终态缓存，非终态刷新失败时可能回退旧状态。
3	容量	worker slots覆盖同时活跃run，槽位不足会排队。
4	连接	核对graph_id、url、鉴权、外部超时。

running只表示状态，不代表已搜索4次或预计几分钟；无证据不要编造进度。

trace用task/thread/run ID串联；追踪不替代运行服务。

本章回顾：能返回，还要能管理

Server承接后台run

check / update / cancel
使用完整task_id。
start 不需要 task_id；
list 不需要 task_id。

update保留thread、
更换run

取消后check；
取消不等于外部操作回滚。

⚠ 注意副作用

async_tasks依赖
持久化恢复

ASGI需异步入口，
容量与鉴权先核对

⚠ 先核对容量与鉴权

**第 7 章：Skills，
可复用的 Agent 能力包**

配套Notebook可验证本地生命周期；

⚠ 本次幻灯片校准是源码与静态检查，不是现场运行结果。